

Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)

Department of CIS, Room F-1030-I

Office hours: Mon 2pm-5pm (in-person and Zoom)

Office phone: 212-220-1490

Borough of Manhattan Community College/CUNY

CSC331 Fall 2025

Data Structures

Week 13:

Lookups using hashing

Professor Kevin Byron

Department of CIS

Borough of Manhattan Community College/CUNY

Reminders

- Program 5 due date
 - 1159pm Fri 12-12-2025
 - late submission receives no credit
- Final exam
 - Section 501 Tue 12-16-2025 – Zoom 520pm-735pm
 - Section 172 Wed 12-17-2025 – In person room M-1001 520pm-735pm
 - Closed book, closed notes, no devices
 - Section 500 Wed 12-17-2025 – In person room N-453 9am-1115am
 - Closed book, closed notes, no devices
- Posted on Brightspace
 - Week 13 lecture slides, practice quiz 3 – not for credit, Sample C++ hashing with chaining (dm456.cpp), Sample C++ range for loop (dm457.cpp)

Searching and hashing algorithms

- Can we construct a search algorithm that is of order less than $\log_2 n$?
- Hashing is a $O(1)$ complexity search algorithm
- Using hashing, the data are organized in a dedicated space called the hash table, denoted HT
- The hash table is stored in contiguous elements in memory or a direct access device

Searching and hashing algorithms

- To determine whether a particular item with a key, say X , is in the table, we apply a function h , called the hash function, to the key X
- We compute $h(X)$, read as h of X
- The function h is typically an arithmetic function and $h(X)$ gives the address of the item X in the hash table
- Suppose that the size of the hash table, HT , is m locations
- Then $0 \leq h(X) < m$

Searching and hashing algorithms

- To determine whether the item with key X is in the table, we look at the entry $HT[h(X)]$ in the hash table
- The items are stored in no particular order
- We consider the following:
 - How do we choose a hash function?
 - How do we organize the data with the help of the hash table?

Searching and hashing algorithms

Hash Functions: Some Examples

Several hash functions are described in the literature. Here we describe some of the commonly used hash functions.

Folding: In folding, the key X is partitioned into parts such that all the parts, except possibly the last parts, are of equal length. The parts are then added, in some convenient way, to obtain the hash address.

Division (Modular arithmetic): In this method, the key X is converted into an integer i_X . This integer is then divided by the size of the hash table to get the remainder, giving the address of X in HT . That is, (in C++)

$$h(X) = i_X \% HTSize;$$

Searching and hashing algorithms

- There are two common ways that data is organized using a hash table:
 - The data may be stored within the hash table, much like an array
 - The data may be stored in linked lists and the hash table holds pointers to those linked lists (also known as chaining)

Searching and hashing algorithms

- The hash table HT may be divided into, say b buckets $HT[0], HT[1], \dots, HT[b - 1]$
- Each bucket is capable of holding, say r items
- $b * r = m$, where m is the size of HT
- When $r = 1$, each bucket can hold one item
- The hash function h maps the key X onto an integer t , that is, $h(X) = t$, such that $0 \leq h(X) \leq b - 1$

Searching and hashing algorithms

Example:

- Suppose there are six students a1, a2, a3, a4, a5, a6 in the Data Structures class

Their IDs are :

- a1: 197354863
- a2: 933185952
- a3: 132489973
- a4: 134152056
- a5: 216500306
- a6: 106500306.

Let:

- k1 = 197354863
- k2 = 933185952
- k3 = 132489973
- k4 = 134152056
- k5 = 216500306
- k6 = 106500306

Searching and hashing algorithms

Suppose that HT denotes the hash table and HT is of size 13 indexed 0, 1, 2, . . . , 12.

- Define the function $h: \{k_1, k_2, k_3, k_4, k_5, k_6\} \rightarrow \{0, 1, 2, \dots, 12\}$ by $h(k_i) = k_i \% 13$.
- (Note that % denotes the mod operator.)

Now

- $h(k_1) = h(197354863) = 197354863 \% 13 = 4$
- $h(k_2) = h(933185952) = 933185952 \% 13 = 10$
- $h(k_3) = h(132489973) = 132489973 \% 13 = 5$
- $h(k_4) = h(134152056) = 134152056 \% 13 = 12$
- $h(k_5) = h(216500306) = 216500306 \% 13 = 9$
- $h(k_6) = h(106500306) = 106500306 \% 13 = 3$

Suppose $HT[b] \leftarrow a$ means "store the data of the student with ID a into $HT[b]$."

Then:

- $HT[4] \leftarrow 197354863$
- $HT[10] \leftarrow 933185952$
- $HT[5] \leftarrow 132489973$
- $HT[12] \leftarrow 134152056$
- $HT[9] \leftarrow 216500306$
- $HT[3] \leftarrow 106500306$

Searching and hashing algorithms

Considerations:

- collision: when two distinct keys that are assigned the same address in the hash table, the second must be stored in a manner that allows retrieval
- record deletion: when a record is deleted, care must be taken to ensure that no other record is left stranded

Searching and hashing algorithms

2 methods for handling collisions:

LINEAR PROBING

Suppose that an item with key X is to be inserted in HT . We use the hash function to compute the index $h(X)$ of this item in HT . Suppose that $h(X) = t$. Then $0 \leq h(X) \leq HTSize - 1$. If $HT[t]$ is empty, we store this item into this array slot. Suppose that $HT[t]$ is already occupied by another item; we have a collision. In linear probing, starting at location t , we search the array sequentially to find the next available array slot.

In linear probing, we assume that the array is circular so that if the lower portion of the array is full, we can continue the search in the top portion of the array. This can be easily accomplished by using the mod operator. That is, starting at t , we check the array locations t , $(t + 1) \% HTSize$, $(t + 2) \% HTSize$, ..., $(t + j) \% HTSize$. This is called the **probe sequence**.

RANDOM PROBING

This method uses a random number generator to find the next available slot. The i th slot in the probe sequence is

$$(h(X) + r_i) \% HTSize$$

where r_i is the i th value in a random permutation of the numbers 1 to $HTSize - 1$. All insertions and searches use the same sequence of random numbers.

Searching and hashing algorithms

Example:

Suppose there are six workers, in a workshop, with IDs 147, 169, 580, 216, 974, and 124. Suppose hash table, HT , is of the size 13, indexed 0, 1, 2, ..., 12. Show how these workers' IDs, in the order given, are inserted in HT using the hashing function $h(k) = k \% 13$. Use linear probing to resolve collision.

... repeat using Random probing to resolve collisions, and using the following random sequence: 6, 3, 10

$h(147) = 147 \% 13 = 4$
 $h(169) = 169 \% 13 = 0$
 $h(580) = 580 \% 13 = 8$
 $h(216) = 216 \% 13 = 8$
collision ... $(8+1) \% 13 = 9$
 $h(974) = 974 \% 13 = 12$
 $h(124) = 124 \% 13 = 7$

0	169
1	
2	
3	
4	147
5	
6	
7	124
8	580
9	216
10	
11	
12	974

$h(147) = 147 \% 13 = 4$
 $h(169) = 169 \% 13 = 0$
 $h(580) = 580 \% 13 = 8$
 $h(216) = 216 \% 13 = 8$
collision ... $(8+6) \% 13 = 1$
 $h(974) = 974 \% 13 = 12$
 $h(124) = 124 \% 13 = 7$

0	169
1	216
2	
3	
4	147
5	
6	
7	124
8	580
9	
10	
11	
12	974

Searching and hashing algorithms

Exercise:

Suppose there are five workers, in a shop, with IDs 909, 185, 657, 116, and 150. Suppose hash table, HT , is of the size 7, indexed 0, 1, 2, ..., 6. Show how these workers' IDs, in the order given, are inserted in HT using the hashing function $h(k) = k \% 7$. Use linear probing to resolve collision.

... repeat using Random probing to resolve collisions, and using the following random sequence: 6, 3, 4

Searching and hashing algorithms

Solutions:

with linear probing

0	657
1	
2	
3	185
4	116
5	150
6	909

with random probing

0	
1	
2	150
3	185
4	116
5	657
6	909

Searching and hashing algorithms

```
// dm457.cpp 12-8-2021 kbyron@bmcc.cuny.edu
// Illustration of range-for loop using C++ code
// ref: https://www.geeksforgeeks.org/range-based-loop-c/
#include <list>
#include <math.h>
#include <iostream>
#include <vector>
using namespace std;
int main() {
    list<int> l = {0, 10, 20, 30, 40, 50};
    for (auto i : l)                // Iterate over list
        cout << i << ' ';
    cout << '\n';
    vector<int> v = {0, 2, 4, 6, 8, 10};
    for (auto i : v)                // Iterate over array
        cout << i << ' ';
    cout << '\n';
    int *p;
    p = new int[6];
    for (int n : {0, 1, 2, 3, 4, 5}) // use braced-init-list
        p[n]=pow(n,2);
    for (int n : {0, 1, 2, 3, 4, 5})
        cout << p[n] << ' ';
    cout << '\n';
}
```

Searching and hashing algorithms

Sample C++ range for loop ... output:

```
de112-551 fy22 >: ./dm457  
0 10 20 30 40 50  
0 2 4 6 8 10  
0 1 4 9 16 25
```

Searching and hashing algorithms

Sample C++ Hashing with chaining:

```
// dm456.cpp 12-8-2021 kbyron@bmcc.cuny.edu
// C++ program to implement hashing with chaining
// ref: https://www.geeksforgeeks.org/c-program-hashing-chaining/
#include<list>
#include<iostream>
using namespace std;
class Hash {
    int buckets;                // No. of buckets
    list<int> *table;            // Pointer to an array containing buckets
public:
    Hash(int V);                // Constructor
    void insertItem(int x);      // inserts a key into hash table
    void deleteItem(int key);    // deletes key from hash table
    int hashFunction(int x) {    // hash function to map key to value
        return (x % buckets);
    }
    void displayHash();
};
```

Searching and hashing algorithms

```
Hash::Hash(int b) {
    buckets = b;
    table = new list<int>[buckets];
}

void Hash::insertItem(int key) {
    int index = hashFunction(key);
    table[index].push_back(key);
}

void Hash::deleteItem(int key) {
    int index = hashFunction(key); // get the hash index of key
    list<int>::iterator i;         // find the key in (index)th list
    for (i = table[index].begin();
         i != table[index].end(); i++) {
        if (*i == key)
            break;
    }
    if (i != table[index].end()) // if key is found, remove it
        table[index].erase(i);
}

void Hash::displayHash() { // function to display hash table
    cout << "Hash Table contents:" << endl;
    for (int i = 0; i < buckets; i++) {
        cout << i;
        for (auto x : table[i])
            cout << " --> " << x;
        cout << endl;
    }
    cout << endl;
}
```

Searching and hashing algorithms

Sample C++ Hashing with chaining:

```
int main() {  
    int a[] = {15, 11, 27, 8, 12}; // array of keys to be mapped  
    int n = sizeof(a)/sizeof(a[0]);  
    Hash h(7); // 7 buckets in hash table  
    for (int i = 0; i < n; i++) // insert the keys into hash table  
        h.insertItem(a[i]);  
    h.displayHash(); // display the Hash table  
    h.deleteItem(12); // delete 12 from hash table  
    h.displayHash(); // display the Hash table  
}
```

Searching and hashing algorithms

Sample C++ Hashing with chaining ... output:

```
de112-612 fy22 >: ./dm456
```

```
Hash Table contents:
```

```
0
```

```
1 --> 15 --> 8
```

```
2
```

```
3
```

```
4 --> 11
```

```
5 --> 12
```

```
6 --> 27
```

```
Hash Table contents:
```

```
0
```

```
1 --> 15 --> 8
```

```
2
```

```
3
```

```
4 --> 11
```

```
5
```

```
6 --> 27
```

Searching and hashing algorithms

Practice: Show output produced by the following:

```
int main() {  
    int a[] = {25, 31, 17, 78, 6, 9};  
    int n = sizeof(a)/sizeof(a[0]);  
    Hash h(7);  
    for (int i = 0; i < n; i++)  
        h.insertItem(a[i]);  
    h.displayHash();  
}
```

Reading Assignment

- Malik chapter 9 – searching and hashing algorithms
- Video on hashing:
 - https://www.youtube.com/watch?v=KyUTuwz_b7zQ
- <https://www.cplusplus.com/reference/list/list/>
- <https://www.geeksforgeeks.org/c-program-hashing-chaining/>